

Дисциплина Языки и системы программирования
(Парадигмы языков программирования)

ОСНОВЫ Haskell

Функции

```
fun :: String -> Int -> String
fun s i = ..
```

Тело функции состоит из 2х частей.

Первая часть это сигнатура функции, которая описывает типы данных аргументов функции и возвращаемые значения. Имя функции и сигнатура разделены символом `::`. Аргументы разделяются символом `->`, последнее значение в сигнатуре это возвращаемый тип функции.

Вторая часть, это непосредственная реализация функции, имя разделено от реализации символом `=`. После имени следуют переменные тип которых будет соответствовать типам из сигнатуры в порядке следования.

Функции

имя :: тип

имя = определение

```
fun s i = ..
```

```
main = fun "abcd" 3
```

Определение функции

```
cube :: Integer -> Integer
```

```
cube n = n * n * n
```

```
cube :: Integer -> Integer
```

```
cube = n * n * n
```

```
cube = n * n * n
```

Язык позволяет описывать функции без конкретного указания типов. Благодаря строгой типизации, компилятор сможет самостоятельно определить типы аргументов, основываясь на том как вызывается функция и аргументы каких типов передаются в функцию.

Такого рода описание удобно при работе из GHCi однако при разработке конкретных программ, лучше описывать сигнатуры функций, т.к. по ним легко можно определить общую логику работы конкретной функции.

Типы данных

Haskell обладает стандартным набором типов, доступных во многих других языках программирования:

Тип	Описание
<code>String</code>	Строки
<code>Int</code>	Целочисленные значения
<code>Float</code>	Значения с плавающей точкой
...	

Абстракция типов

```
shift :: Signal -> Integer -> Signal
```

Signal свой тип данных

Integer стандартный тип данных

shift - сдвигает сигнал на некоторое количество отсчетов

Свои типы данных

```
type A = B
```

<- синоним

```
data Bool = True | False
```

<- новый тип данных

Свои типы данных

```
data Bool = True | False
```

все что после "=" конструкторы значений

```
data Freq = Freq Int Int
```

- обертка, содержит 2 значения

Промежуточный итог

- Функции более высокоуровневые => работа с прикладными понятиями
- Отсутствие побочных эффектов
- Определения = уравнение => можно говорить о свойствах и доказательствах
- Удобство рефакторинга
- Легкость в организации параллельных вычислений

Prelude

Вместе с компилятором поставляется базовый пакет, который содержит необходимый набор различных функций, которые могут понадобиться при повседневной разработке

**ядро платформы с набором
предопределенных функций на все случаи
жизни**

Коллекции

Коллекции необходимы в любом языке программирования, т.к. мы всегда работаем с некими наборами данных. Следовательно у нас должны быть возможности представлять их и взаимодействовать с ними. В Haskell для этого используются списки, которые обозначаются через []. Коллекция целых чисел [3, 1, 41, 592, 2653], коллекция строк ["aa", "bb", "cc"], даже строка это коллекция символов "aa" - ['a', 'a'], Данная структура данных тоже полиморфна и мы можем использовать любые типы для составления списков.

Коллекции

Списки (Lists) [], [1, 2, 3, 4], ["abcd", "abcde"]

String = [Char]

Абстракция списков (List comprehensions)

`[1..10]`

`[i, j | i <- [1..2] , j <- [1..5]]`

`[2*x | x <- [1..], x^2 > 3]`

Функции списков

`head, tail, init, last, ...`

напиши свой

Операции над коллекциями

Никаких явных циклов

`map (a->b)->[a]->[b]`

`foldl (a -> b -> a) -> a -> [b] -> a`

`foldr (a -> b -> b) -> b -> [a] -> b`

Рекурсии и абстракции

Hoogle

Для того, чтоб ориентироваться во всем многообразии функциональных возможностей, и не переизобретать уже готовый функционал существует сервис поиска функций, как по именами так и по их сигнатурам.

Если вы подумываете над тем, чтоб реализовать функцию с какой-то сигнатурой типов, например `String->Int->String` лучше вбить ее в Hoogle и убедиться, что она уже существует.

Конечно, для образовательных целей бывает полезно переизобретать некоторые части функциональности, для того чтоб понять как они работают. Hoogle выполняет поиск не только по Prelude но и по всему Hackage, так что помогает быстро разобраться какой пакет используется и какой конкретный модуль в этом пакете, если необходимо понять что происходит.

<https://www.haskell.org/hoogle/>

Простые примеры

Рассмотрим несколько примеров, для того, чтоб закрепить полученную информацию.

Функции это основа нашей программы. В Haskell все происходит путем взаимодействия с функциями, это позволяет производить грамотную декомпозицию(дробление) прикладной области и реализовывать код эффективно. Давайте рассмотрим простой пример и изучим из чего состоит функция

```
-- | Применить дважды
--
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

В данном примере, функция которая принимает в качестве аргумента другую функцию, какую-то переменную и применяет эту функцию к переменной 2 раза.



Первая строка это сигнатура функции, которая описывает типы данных аргументов функции и возвращаемые значения. Имя функции и сигнатура разделены символом `::`. Аргументы разделяются символом `->`, последнее значение в сигнатуре это возвращаемый тип функции.

Вторая строка, это непосредственная реализация функции, имя разделено от реализации символом `=`. После имени следуют переменные тип которых будет соответствовать типам из сигнатуры в порядке следования. После равно непосредственная функциональность.

При использовании данной функции, система сама поймет какой тип используется, в сигнатуре типа мы используем **a** это параметризованный тип, мы используем его когда нет необходимости конкретно указать это **Int**, **String** или какой-то свой тип данных. Пример использования:

```
main = applyTwice (^2) 3
```

```
81
```

В данном случае функция **(^2)** это возведение в квадрат, **3** это число

```
main = applyTwice (++ " Step") "IT"
```

```
"IT Step Step"
```

В данном случае функция **(++ " Step")** конкатенация начальной строки со строкой **"Step"**, **"IT"** значение начальной строки

Этот механизм доступен благодаря полиморфизму и выводу типов, чем-то напоминает механизм **generics** в **Java** или **C#**, но намного более простой в использовании.

Выводы

Язык общего назначения

- Высоконагруженные веб-порталы (Yesod, Snap)
- Компиляторы
- Высокочастотный трейдинг
- Распределенные вычисления
- Параллельные вычисления

Вписывается в архитектуру любого приложения

- FFI (Foreign Functions Interface)
- Java-Bridge <https://hackage.haskell.org/package/java-bridge>